Tópico 10.

Prompt:
Olá gemini, seguinte, eu possuo as seguintes classes em Java:

```
package compartilhado;
import java.io.*;

public class Ponto2D implements Serializable {
   private int x;
   private int y;

   public Ponto2D(int x, int y){
      this.x = x;
      this.y = y;
   }

   public int getX(){
      return this.x;
   }

   public int getY(){
      return this.y;
   }

   public void setX(int x){
      this.x = x;
   }

   public void setY(int y){
      this.y = y;
   }
}

package compartilhado;
import java.io.*;

public class Pinguim implements Serializable{
   private int id;
   // private Ponto2D posicaoAtual;
   private Ponto2D posicaoAlvo; //Para o servidor Alvo é a posição atual
   private boolean estaDancando;
```

```
private String cor;

public Pinguim(int id, int x_Alvo, int y_Alvo, boolean estaDancando, String cor){
    this.id = id;
    // this.posicaoAtual = new Ponto2D(x_Atual, y_Atual);
    this.posicaoAlvo = new Ponto2D(x_Alvo, y_Alvo);
    this.estaDancando = estaDancando;
    this.cor = cor;
}

public int getId(){
    return this.id;
}

public void setId(int id){
    this.id = id;
}

public boolean getDancando(){
    return this.estaDancando;
}

public void setDancando(boolean dancando){
    this.estaDancando = dancando;
}

public void dancar(){
    this.estaDancando = !this.estaDancando;
}

public String getCor(){
    return this.cor;
}

public void setCor(String cor){
    this.cor = cor;
}

// public int getXAtual(){
//  return posicaoAtual.getX();
// }

// public int getYAtual(){
//  return posicaoAtual.getY();
```

```
    // }

    public int getXAlvo(){
        return posicaoAlvo.getX();
    }

    public int getYAlvo(){
        return posicaoAlvo.getY();
    }

    // public void setXAtual(int x){
    //  this.posicaoAtual.setX(x);
    // }

    // public void setYAtual(int y){
    //  this.posicaoAtual.setY(y);
    // }

    public void setXAlvo(int x){
        this.posicaoAlvo.setX(x);
    }

    public void setYAlvo(int y){
        this.posicaoAlvo.setY(y);
    }
}
```

Resposta:
Excelente\! As suas classes `Ponto2D` e `Pinguim` estão bem estruturadas e implementam corretamente a interface `Serializable`, o que é essencial para a **serialização** e **desserialização** em Java.

Com base nas classes fornecidas, o seu objetivo principal será enviar e receber objetos `Pinguim` (que internamente contêm um objeto `Ponto2D`) através de *streams* (como em comunicação de rede via *sockets* ou escrita em arquivo).

Abaixo, apresento um exemplo completo de como você pode serializar (escrever) e desserializar (ler) um objeto `Pinguim` usando `ObjectOutputStream` e `ObjectInputStream`, simulando o fluxo de dados em memória, que é o mesmo princípio aplicado em comunicação de rede ou E/S de arquivos.

Exemplo de Serialização e Desserialização de `Pinguim`

Este exemplo usa `ByteArrayOutputStream` e `ByteArrayInputStream` para simular a escrita e leitura dos bytes do objeto `Pinguim` em memória.

1\. Classe de Exemplo (`TesteSerializacao`)

```java
package principal;

import compartilhado.Pinguim;
import java.io.*;

public class TesteSerializacao {

    public static void main(String[] args) {
        // 1. Criação do Objeto Original
        Pinguim pinguimOriginal = new Pinguim(
            101,            // id
            50, 75,         // x_Alvo, y_Alvo
            true,           // estaDancando
            "Azul"          // cor
        );

        System.out.println("--- Objeto Original ---");
        System.out.println("ID: " + pinguimOriginal.getId());
        System.out.println("Posição Alvo: (" + pinguimOriginal.getXAlvo() + ", " + pinguimOriginal.getYAlvo() + ")");
        System.out.println("Está Dançando: " + pinguimOriginal.getDancando());
        System.out.println("Cor: " + pinguimOriginal.getCor());

        // Para serialização, usaremos streams de byte em memória (simula rede ou arquivo)
        ByteArrayOutputStream byteOut = new ByteArrayOutputStream();

        // Objeto que será desserializado (inicializado como null)
        Pinguim pinguimDesserializado = null;

        // --- Serialização (Escrita do Objeto) ---

        try (ObjectOutputStream objectOut = new ObjectOutputStream(byteOut)) {

            // O método writeObject escreve o objeto Pinguim, e, de forma
            // transparente, o objeto Ponto2D referenciado dentro dele.
            objectOut.writeObject(pinguimOriginal);
            System.out.println("\n✅ Serialização do objeto Pinguim realizada com sucesso.");
```

```
    } catch (IOException e) {
        System.err.println("Erro na serialização: " + e.getMessage());
        e.printStackTrace();
        return; // Termina se houver erro na escrita
    }

    // Os bytes serializados
    byte[] dadosSerializados = byteOut.toByteArray();
      System.out.println("Tamanho dos dados serializados: " + dadosSerializados.length + "
bytes");

    // --- Desserialização (Leitura do Objeto) ---

    try (ByteArrayInputStream byteIn = new ByteArrayInputStream(dadosSerializados);
         ObjectInputStream objectIn = new ObjectInputStream(byteIn)) {

        // O método readObject retorna um Object, que deve ser
        // convertido (cast) para a classe Pinguim.
        pinguimDesserializado = (Pinguim) objectIn.readObject();
                System.out.println("\n✅ Desserialização do objeto Pinguim realizada com
sucesso.");

    } catch (IOException e) {
        System.err.println("Erro de E/S na desserialização: " + e.getMessage());
        e.printStackTrace();
    } catch (ClassNotFoundException e) {
                 System.err.println("Classe não encontrada durante a desserialização: " +
e.getMessage());
        e.printStackTrace();
    }

    // --- Verificação do Objeto Desserializado ---

    if (pinguimDesserializado != null) {
        System.out.println("\n--- Objeto Desserializado ---");
        System.out.println("ID: " + pinguimDesserializado.getId());
           System.out.println("Posição Alvo: (" + pinguimDesserializado.getXAlvo() + ", " +
pinguimDesserializado.getYAlvo() + ")");
        System.out.println("Está Dançando: " + pinguimDesserializado.getDancando());
        System.out.println("Cor: " + pinguimDesserializado.getCor());

        // Verifica se o objeto foi recriado com sucesso (comparação de atributos)
        if (pinguimOriginal.getId() == pinguimDesserializado.getId() &&
           pinguimOriginal.getCor().equals(pinguimDesserializado.getCor())) {
```

```
            System.out.println("\nSucesso! O objeto foi restaurado corretamente.");
        }
    }
  }
}
```

Observações Importantes sobre Serialização

1. **Interface `Serializable`**: Você implementou corretamente a interface de marcação `Serializable` tanto na classe **`Pinguim`** quanto na classe **`Ponto2D`**. É crucial que todos os objetos que serão serializados, *e todos os objetos referenciados por eles*, implementem essa interface. Se `Ponto2D` não implementasse `Serializable`, a serialização de `Pinguim` falharia.
2. **`ObjectOutputStream` e `ObjectInputStream`**:
   * **Serialização**: Use `ObjectOutputStream.writeObject(Object obj)` para transformar o estado do objeto em uma sequência de bytes.
   * **Desserialização**: Use `ObjectInputStream.readObject()` para reconstruir o objeto a partir da sequência de bytes. O retorno é um `Object`, então é necessário fazer o *cast* explícito para a classe correta (`Pinguim`).
3. **Encadeamento de *Streams***:
   * Serialização: O `ObjectOutputStream` é "encadeado" em um *stream* de saída de baixo nível (como `FileOutputStream` para um arquivo, `Socket.getOutputStream()` para rede, ou `ByteArrayOutputStream` para memória).
   * Desserialização: O `ObjectInputStream` é encadeado no *stream* de entrada correspondente (`FileInputStream`, `Socket.getInputStream()`, ou `ByteArrayInputStream`).
4. **`transient`**: Se houvesse algum campo (como, por exemplo, um *stream* ou um *cache*) que **não** devesse ser serializado, você poderia marcá-lo com a palavra-chave **`transient`**.
   * *Exemplo:* `private transient int contadorTemporario;`

Prompt: Não quero que você se atente a serialização, ou ao envio por meio de sockets agora. Primeiro quero fazer algo básico, gostaria de utilizar as bibliotecas awt e swing de Java para criar um jogo simples, no qual minha tela é totalmente branca, e existe apenas uma instância de Pinguim no centro dela, que terá como representação gráfica um sprite que adicionarei posteriormente, nomeado de voltante.png.

Resposta: É um excelente começo\! Para criar um jogo simples em Java Swing com uma imagem móvel, usaremos as seguintes técnicas:

1. **`JFrame`**: A janela principal do aplicativo.

2. **`JPanel`**: Um painel personalizado para desenhar (`paintComponent`) o fundo e a imagem do pinguim.

3. **`ImageIcon`**: Para carregar o *sprite* (`voltante.png`).

4. **`Pinguim`**: Sua classe, para gerenciar a posição (`posicaoAlvo`).

O código a seguir cria a janela, desenha o fundo branco e posiciona a imagem do pinguim no centro, lendo a posição da sua classe `Pinguim`.

Estrutura do Projeto

Certifique-se de que a sua estrutura de pastas está correta, principalmente a localização do arquivo de imagem:

```

seu_projeto/
```

```
├── src/
│   ├── compartilhado/
│   │   ├── Ponto2D.java
│   │   └── Pinguim.java
│   └── principal/
│       └── JogoPinguim.java (Nova Classe)
└── voltante.png  <-- O arquivo de imagem deve estar aqui (ou no classpath)
```

1\. Classe `JogoPinguim` (Principal)

Esta classe cria a janela (`JFrame`) e o painel de desenho (`PainelJogo`).

```java
package principal;
```

```
import compartilhado.Pinguim;

import java.awt.*;

import javax.swing.*;



public class JogoPinguim {



    public static void main(String[] args) {

        // Garantir que a GUI é iniciada na thread de Eventos do Swing (EDT)

        SwingUtilities.invokeLater(() -> {



            // Cria a instância do Pinguim no centro inicial da tela

            // Assumindo tela de 800x600, o centro é (400, 300)

            Pinguim pinguim = new Pinguim(1, 400, 300, false, "Preto");
```

```
        // 1. Configuração da Janela Principal (JFrame)

        JFrame frame = new JFrame("Pinguim Game (Voltante Edition)");

        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        frame.setSize(800, 600); // Tamanho da janela

        frame.setLocationRelativeTo(null); // Centraliza a janela na tela



        // 2. Criação e Adição do Painel do Jogo

        PainelJogo painelJogo = new PainelJogo(pinguim, "voltante.png");

        frame.add(painelJogo);



        // 3. Exibir a Janela

        frame.setVisible(true);

    });

  }

}
```

```

### 2\. Classe `PainelJogo` (Onde o Desenho Acontece)

Esta classe estende `JPanel` e substitui o método `paintComponent` para realizar o desenho do fundo e do Pinguim.

```java
package principal;

import compartilhado.Pinguim;
import java.awt.*;
import javax.swing.*;
import java.io.File;

// Classe que cuidará do desenho dos elementos do jogo
```
```

```
class PainelJogo extends JPanel {

    private static final long serialVersionUID = 1L;



    private Pinguim pinguim;

    private Image spritePinguim;

    private final int LARGURA_SPRITE = 64; // Largura do seu sprite (ajuste se necessário)

    private final int ALTURA_SPRITE = 64;  // Altura do seu sprite (ajuste se necessário)



    public PainelJogo(Pinguim pinguim, String caminhoSprite) {

        this.pinguim = pinguim;



        // Carrega a imagem do pinguim

        try {

            // Tenta carregar a imagem do classpath (melhor para aplicações JAR)

            ImageIcon icon = new
ImageIcon(getClass().getClassLoader().getResource(caminhoSprite));
```

```
        spritePinguim = icon.getImage();



        // Verifica se a imagem foi carregada corretamente (pode ser necessário o caminho
absoluto

        // se estiver rodando o código fora de um ambiente de IDE configurado)

        if (spritePinguim == null) {

            System.err.println("Aviso: Imagem não encontrada no Classpath. Tentando
caminho direto.");

            // Tenta carregar do caminho direto (pode depender de onde o código é executado)

            spritePinguim = new ImageIcon(caminhoSprite).getImage();

        }



    } catch (Exception e) {

        System.err.println("Erro ao carregar o sprite do pinguim: " + e.getMessage());

        e.printStackTrace();

    }
```

```
    // Define o fundo branco

    setBackground(Color.WHITE);

}



// Este é o método mais importante para o desenho em Swing

@Override

protected void paintComponent(Graphics g) {

    super.paintComponent(g); // Limpa e pinta o fundo (que definimos como branco)



    // Desenha a imagem do Pinguim

    if (spritePinguim != null) {



        // As coordenadas (x, y) no Pinguim representam o centro do sprite

        int x_posicao = pinguim.getXAlvo() - (LARGURA_SPRITE / 2);

        int y_posicao = pinguim.getYAlvo() - (ALTURA_SPRITE / 2);
```

```
        // Desenha a imagem na tela

        // g.drawImage(imagem, x, y, largura, altura, observador)

        g.drawImage(spritePinguim,

                x_posicao,

                y_posicao,

                LARGURA_SPRITE,

                ALTURA_SPRITE,

                this);

    } else {

        // Se a imagem não carregar, desenha um círculo simples

        g.setColor(Color.BLUE);

        g.fillOval(pinguim.getXAlvo() - 32, pinguim.getYAlvo() - 32, 64, 64);

        g.setColor(Color.BLACK);

        g.drawString("Pinguim (Sem Sprite)", pinguim.getXAlvo() - 50, pinguim.getYAlvo()
- 35);
```

```
        }

    }

}
```

Como Executar (Passos Importantes)

1. **Salve os Arquivos**: Salve as classes `Ponto2D.java` e `Pinguim.java` no pacote `compartilhado`. Salve `JogoPinguim.java` e `PainelJogo.java` (se você as separar) no pacote `principal`.

2. **Coloque a Imagem**: Coloque o arquivo **`voltante.png`** na raiz da sua pasta do projeto ou na pasta onde o seu projeto Java espera encontrar recursos (geralmente a pasta `src` ou a raiz do seu *classpath*).

3. **Compile e Execute**: Rode a classe `JogoPinguim`. Você verá uma janela branca com o seu *sprite* `voltante.png` centralizado.

Prompt: Perfeito, funcionou. Agora, eu gostaria de ser capaz de movimentar essa instância de pinguim pela minha tela utilizando cliques do mouse.

Resposta: Excelente\! Mover o pinguim com cliques do mouse é um passo fundamental para transformar a aplicação em um jogo interativo.

Para isso, usaremos o **`MouseListener`** e o **`MouseMotionListener`** do Java Swing. Faremos as seguintes alterações no seu código:

1. A classe `PainelJogo` implementará o `MouseListener` para capturar os cliques.

2. No método `mouseClicked`, atualizaremos as coordenadas de **`posicaoAlvo`** do objeto `Pinguim` com base na posição do clique do mouse.

3. Chamaremos `repaint()` para forçar o redesenho do pinguim na nova posição.

1\. Atualize a classe `PainelJogo`

Altere a classe `PainelJogo.java` para implementar `MouseListener` e adicionar a lógica de movimento.

```java

package principal;

import compartilhado.Pinguim;
```

```
import java.awt.*;

import javax.swing.*;

import java.awt.event.MouseEvent;

import java.awt.event.MouseListener;



// A classe PainelJogo agora implementa MouseListener

class PainelJogo extends JPanel implements MouseListener {

    private static final long serialVersionUID = 1L;



    private Pinguim pinguim;

    private Image spritePinguim;

    private final int LARGURA_SPRITE = 64;

    private final int ALTURA_SPRITE = 64;



    public PainelJogo(Pinguim pinguim, String caminhoSprite) {
```

```
        this.pinguim = pinguim;



        // --- CÓDIGO DE CARREGAMENTO DO SPRITE (MANTIDO INALTERADO) ---

        try {

            ImageIcon icon = new
ImageIcon(getClass().getClassLoader().getResource(caminhoSprite));

            spritePinguim = icon.getImage();



            if (spritePinguim == null) {

                System.err.println("Aviso: Imagem não encontrada no Classpath. Tentando
caminho direto.");

                spritePinguim = new ImageIcon(caminhoSprite).getImage();

            }



        } catch (Exception e) {

            System.err.println("Erro ao carregar o sprite do pinguim: " + e.getMessage());

            e.printStackTrace();
```

```
    }

    // ----------------------
```

```
            int x_posicao = pinguim.getXAlvo() - (LARGURA_SPRITE / 2);

            int y_posicao = pinguim.getYAlvo() - (ALTURA_SPRITE / 2);



            g.drawImage(spritePinguim, x_posicao, y_posicao, LARGURA_SPRITE,
ALTURA_SPRITE, this);



            // Opcional: Desenha um pequeno círculo onde o usuário clicou (Alvo)

            g.setColor(Color.RED);

            g.fillOval(pinguim.getXAlvo() - 3, pinguim.getYAlvo() - 3, 6, 6);

        } else {

            g.setColor(Color.BLUE);

            g.fillOval(pinguim.getXAlvo() - 32, pinguim.getYAlvo() - 32, 64, 64);

            g.setColor(Color.BLACK);

            g.drawString("Pinguim (Sem Sprite)", pinguim.getXAlvo() - 50, pinguim.getYAlvo()
- 35);

        }

    }
```

```
    // ----------------------
```

```
    repaint();



    System.out.println("Pinguim movido para: (" + novoX + ", " + novoY + ")");

}



// Os métodos a seguir fazem parte da interface MouseListener,

// mas não serão usados no momento, então ficam vazios.

@Override

public void mousePressed(MouseEvent e) {}



@Override

public void mouseReleased(MouseEvent e) {}



@Override

public void mouseEntered(MouseEvent e) {}
```

```
    @Override

    public void mouseExited(MouseEvent e) {}

}
```

2\. Ações de Teste

1. Compile e execute a classe `JogoPinguim`.

2. O pinguim aparecerá no centro da tela.

3. Clique em qualquer lugar da tela. O pinguim deve **saltar instantaneamente** para o local do clique.

Próxima Etapa: Se o objetivo for fazer o pinguim **se mover suavemente** até o alvo em vez de saltar, o próximo passo seria implementar um *loop* de jogo e usar *Timers* do Swing para animação.

Prompt: Perfeito, funcionou muito bem, o que eu gostaria agora é de justamente fazer essa movimentação suave. Eu possuo a seguinte classe de pinguim, que agora possui atributos de posição atual e posição alvo, isso é para aplicar minha ideia, esta é a que segue: a cada clique

do mouse a posição alvo será alterada e a posição atual será alterada de forma suave para atingir a posição alvo, o desenho do sprite na tela deverá respeitar a posição atual.

```
package cliente;

import java.io.*;

import compartilhado.*;


public class PinguimRender extends Pinguim{

    private transient Ponto2D posicaoAtual;


    // public PinguimRender(int id, int x_Atual, int y_Atual, int x_Alvo, int y_Alvo, boolean
estaDancando, String cor){

    //  super(id, x_Alvo, y_Alvo, estaDancando, cor);

    //  this.posicaoAtual = new Ponto2D(x_Atual, y_Atual);

    // }
```

```
public PinguimRender(Pinguim pinguim) {

    super(pinguim.getId(),

    pinguim.getXAlvo(),

    pinguim.getYAlvo(),

    pinguim.getDancando(),

    pinguim.getCor()

    );



    this.posicaoAtual = new Ponto2D(pinguim.getXAlvo(), pinguim.getYAlvo());



}



public void setXAtual(int x){

    this.posicaoAtual.setX(x);

}
```

```
    public void setYAtual(int y){

        this.posicaoAtual.setY(y);

    }



}
```

Gostaria que removesse esse quadrado vermelho no centro do sprite também

Resposta: Ótimo\! Sua abordagem de usar uma classe de renderização (`PinguimRender`) que estende a classe original e adiciona a `posicaoAtual` (marcada corretamente como `transient`, já que é um estado local do cliente) é perfeita para implementar a movimentação suave.

Para implementar a animação suave e remover o quadrado vermelho, faremos as seguintes modificações:

1. **Ajuste na `PinguimRender`**: Adicionar métodos *getter* para `posicaoAtual` e o método de lógica de movimento.

2. **Atualização no `PainelJogo`**:

* Remover o código do quadrado vermelho.

* Mudar o desenho para usar `getXAtual()` e `getYAtual()`.

* Implementar um `javax.swing.Timer` para criar o *loop* de jogo e mover o pinguim 60 vezes por segundo.

3. **Atualização no `JogoPinguim`**: Usar a nova classe `PinguimRender`.

1\. Atualização da Classe `PinguimRender`

Adicione os *getters* para `posicaoAtual` e o novo método `atualizarPosicao()`.

```java
package cliente;

import java.io.*;

import compartilhado.Ponto2D; // Importar Ponto2D, se não estiver no mesmo pacote

public class PinguimRender extends compartilhado.Pinguim {
```

// É importante usar o nome completo ou importar a classe base Pinguim do pacote compartilhado.

private transient Ponto2D posicaoAtual;

private final double VELOCIDADE_DE_MOVIMENTO = 0.1; // Fator de suavização (0.0 a 1.0)

// Valores menores (ex: 0.05) deixam o movimento mais lento/suave.

public PinguimRender(compartilhado.Pinguim pinguim) {

super(pinguim.getId(),

pinguim.getXAlvo(),

pinguim.getYAlvo(),

pinguim.getDancando(),

pinguim.getCor()

);

// Inicializa a Posição Atual no mesmo lugar que o Alvo

```
    this.posicaoAtual = new Ponto2D(pinguim.getXAlvo(), pinguim.getYAlvo());

}



// Novos Getters para Posição ATUAL

public int getXAtual(){

    return this.posicaoAtual.getX();

}



public int getYAtual(){

    return this.posicaoAtual.getY();

}



// Setters (mantidos)

public void setXAtual(int x){

    this.posicaoAtual.setX(x);
```

```
}

public void setYAtual(int y){
    this.posicaoAtual.setY(y);
}

/**
 * Move suavemente a posicaoAtual em direção à posicaoAlvo.
 */
public void atualizarPosicao() {
    // Posições (usamos double para cálculo preciso)
    double currentX = this.posicaoAtual.getX();
    double currentY = this.posicaoAtual.getY();
    int targetX = super.getXAlvo();
    int targetY = super.getYAlvo();
```

```
// Calcula a diferença (distância)

double dx = targetX - currentX;

double dy = targetY - currentY;



// Verifica se o pinguim está "próximo o suficiente" do alvo

if (Math.abs(dx) < 1 && Math.abs(dy) < 1) {

    // Se estiver perto, define a posição atual como o alvo para evitar tremores

    this.posicaoAtual.setX(targetX);

    this.posicaoAtual.setY(targetY);

    return;

}



// Calcula o movimento com base no fator de velocidade (interpolação linear)

double moveX = dx * VELOCIDADE_DE_MOVIMENTO;
```

```
        double moveY = dy * VELOCIDADE_DE_MOVIMENTO;



        // Aplica o movimento e atualiza a posição atual

        this.posicaoAtual.setX((int) (currentX + moveX));

        this.posicaoAtual.setY((int) (currentY + moveY));

    }

}
```

2\. Atualização da Classe `PainelJogo` (O *Game Loop*)

Implementaremos o `Timer` para chamar o método `atualizarPosicao()` repetidamente e forçar o redesenho.

```java

package principal;
```

```
import cliente.PinguimRender; // Agora usamos PinguimRender

import java.awt.*;

import javax.swing.*;

import java.awt.event.MouseEvent;

import java.awt.event.MouseListener;

import java.awt.event.ActionListener;

import java.io.File;



class PainelJogo extends JPanel implements MouseListener, ActionListener {

    private static final long serialVersionUID = 1L;



    // NOTA: Mudamos o tipo de Pinguim para PinguimRender

    private PinguimRender pinguim;

    private Image spritePinguim;
```

```
    private final int LARGURA_SPRITE = 64;

    private final int ALTURA_SPRITE = 64;



    // **NOVO**: Timer para controlar a animação (Game Loop)

    private Timer gameTimer;

    private final int FPS = 60; // Frames por segundo

    private final int DELAY_MS = 1000 / FPS; // Intervalo de 16ms para 60 FPS



    public PainelJogo(PinguimRender pinguim, String caminhoSprite) {

        this.pinguim = pinguim;



        // --- CÓDIGO DE CARREGAMENTO DO SPRITE ---

        try {

            ImageIcon icon = new
ImageIcon(getClass().getClassLoader().getResource(caminhoSprite));

            spritePinguim = icon.getImage();
```

```
        if (spritePinguim == null) {

            System.err.println("Aviso: Imagem não encontrada no Classpath. Tentando
caminho direto.");

            spritePinguim = new ImageIcon(caminhoSprite).getImage();

        }

    } catch (Exception e) {

        System.err.println("Erro ao carregar o sprite do pinguim: " + e.getMessage());

        e.printStackTrace();

    }

    // ---------------------------------------------------



    setBackground(Color.WHITE);

    this.addMouseListener(this);



    // **NOVO**: Inicializa e inicia o Timer
```

```
    gameTimer = new Timer(DELAY_MS, this);

    gameTimer.start();

}



@Override

protected void paintComponent(Graphics g) {

    super.paintComponent(g);



    if (spritePinguim != null) {



        // **IMPORTANTE**: Usamos a posição ATUAL para desenhar

        int x_atual = pinguim.getXAtual();

        int y_atual = pinguim.getYAtual();



        int x_posicao = x_atual - (LARGURA_SPRITE / 2);
```

```
            int y_posicao = y_atual - (ALTURA_SPRITE / 2);



            g.drawImage(spritePinguim, x_posicao, y_posicao, LARGURA_SPRITE,
ALTURA_SPRITE, this);



            // REMOVEMOS o quadrado/círculo vermelho de debug.



        } else {

            // fallback (usa a posição atual para o círculo)

            g.setColor(Color.BLUE);

            g.fillOval(pinguim.getXAtual() - 32, pinguim.getYAtual() - 32, 64, 64);

            g.setColor(Color.BLACK);

            g.drawString("Pinguim (Sem Sprite)", pinguim.getXAtual() - 50,
pinguim.getYAtual() - 35);

        }

    }
```

```
    // *************** MÉTODOS DA INTERFACE MOUSELISTENER ***************



    @Override

    public void mouseClicked(MouseEvent e) {

        // 1. Obtém as coordenadas do clique

        int novoX = e.getX();

        int novoY = e.getY();



        // 2. Atualiza a **POSIÇÃO ALVO** do Pinguim

        pinguim.setXAlvo(novoX);

        pinguim.setYAlvo(novoY);



        // A posição ATUAL será alcançada gradualmente pelo Timer

        System.out.println("Alvo definido para: (" + novoX + ", " + novoY + ")");

    }
```

```
    // (Outros métodos MouseListener permanecem vazios...)

    @Override public void mousePressed(MouseEvent e) {}

    @Override public void mouseReleased(MouseEvent e) {}

    @Override public void mouseEntered(MouseEvent e) {}

    @Override public void mouseExited(MouseEvent e) {}



    // *************** MÉTODO DA INTERFACE ACTIONLISTENER (O Game Loop)
***************



    @Override

    public void actionPerformed(java.awt.event.ActionEvent e) {

        // 1. Atualiza a lógica do Pinguim

        pinguim.atualizarPosicao();



        // 2. Força o redesenho (chama paintComponent)
```

```
        repaint();

    }

}
```

3\. Atualização da Classe `JogoPinguim`

Certifique-se de que a classe principal cria uma instância de `PinguimRender` em vez de `Pinguim`.

```java
package principal;

import compartilhado.Pinguim;

import cliente.PinguimRender; // Importar a nova classe

import java.awt.*;
```

```
import javax.swing.*;



public class JogoPinguim {



    public static void main(String[] args) {

        SwingUtilities.invokeLater(() -> {



            // 1. Cria a base Pinguim (com a posição inicial Alvo)

            compartilhado.Pinguim basePinguim = new Pinguim(1, 400, 300, false, "Preto");



            // 2. Cria a instância de renderização/movimentação a partir da base

            PinguimRender pinguimRender = new PinguimRender(basePinguim);



            // 1. Configuração da Janela Principal (JFrame)

            JFrame frame = new JFrame("Pinguim Game (Voltante Edition)");
```

```
            frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

            frame.setSize(800, 600);

            frame.setLocationRelativeTo(null);



            // 2. Criação e Adição do Painel do Jogo, passando o PinguimRender

            PainelJogo painelJogo = new PainelJogo(pinguimRender, "voltante.png");

            frame.add(painelJogo);



            // 3. Exibir a Janela

            frame.setVisible(true);

        });

    }

}
```

Ao executar o `JogoPinguim` agora, o clique do mouse definirá um novo `posicaoAlvo`, e o `gameTimer` (via `actionPerformed`) atualizará a `posicaoAtual` em pequenos passos, resultando em um movimento suave e agradável do seu *sprite* até o ponto do clique.

Prompt: Agora eu gostaria que embaixo do sprite do pinguim existisse uma sombra, pode ser uma elipse preta de baixa opacidade.

Resposta: Excelente adição\! Adicionar uma sombra dará profundidade e realismo à movimentação do seu pinguim.

Para desenhar uma elipse preta com baixa opacidade, usaremos a classe **`Graphics2D`** (que é uma versão melhorada de `Graphics`) e a classe **`AlphaComposite`** para gerenciar a transparência.

Faremos as seguintes modificações no método `paintComponent` da classe `PainelJogo`:

1. Obteremos um objeto `Graphics2D` do objeto `Graphics`.

2. Definiremos a cor preta.

3. Usaremos `AlphaComposite` para definir a opacidade (transparência).

4. Desenho da elipse.

5. **IMPORTANTE**: Restauraremos o `AlphaComposite` original para que o pinguim seja desenhado totalmente opaco (sem transparência).

Atualização da Classe `PainelJogo`

Abaixo está o código revisado do `PainelJogo.java`:

```java
package principal;

import cliente.PinguimRender;
import java.awt.*;
import javax.swing.*;
import java.awt.event.MouseEvent;
import java.awt.event.MouseListener;
import java.awt.event.ActionListener;
import java.io.File;
```

```
class PainelJogo extends JPanel implements MouseListener, ActionListener {

    private static final long serialVersionUID = 1L;



    private PinguimRender pinguim;

    private Image spritePinguim;

    private final int LARGURA_SPRITE = 64;

    private final int ALTURA_SPRITE = 64;



    // Dimensões da sombra

    private final int SHADOW_WIDTH = (int) (LARGURA_SPRITE * 0.8); // 80% da
largura do sprite

    private final int SHADOW_HEIGHT = (int) (ALTURA_SPRITE * 0.25); // 25% da altura
do sprite

    private final float SHADOW_OPACITY = 0.4f; // 40% de opacidade (0.0f = transparente,
1.0f = opaco)
```

```
    private Timer gameTimer;

    private final int FPS = 60;

    private final int DELAY_MS = 1000 / FPS;



    public PainelJogo(PinguimRender pinguim, String caminhoSprite) {

        this.pinguim = pinguim;



        // --- CÓDIGO DE CARREGAMENTO DO SPRITE (MANTIDO INALTERADO) ---

        try {

            ImageIcon icon = new
ImageIcon(getClass().getClassLoader().getResource(caminhoSprite));

            spritePinguim = icon.getImage();



            if (spritePinguim == null) {

                System.err.println("Aviso: Imagem não encontrada no Classpath. Tentando
caminho direto.");

                spritePinguim = new ImageIcon(caminhoSprite).getImage();
```

```
        }
    } catch (Exception e) {
        System.err.println("Erro ao carregar o sprite do pinguim: " + e.getMessage());
        e.printStackTrace();
    }
    // ---------------------------------------------------


    setBackground(Color.WHITE);
    this.addMouseListener(this);


    gameTimer = new Timer(DELAY_MS, this);
    gameTimer.start();
}


@Override
```

```
protected void paintComponent(Graphics g) {

    super.paintComponent(g);



    // 1. Converte o objeto Graphics para Graphics2D (necessário para transparência)

    Graphics2D g2d = (Graphics2D) g;



    int x_atual = pinguim.getXAtual();

    int y_atual = pinguim.getYAtual();



    // As coordenadas de desenho para o sprite

    int x_sprite = x_atual - (LARGURA_SPRITE / 2);

    int y_sprite = y_atual - (ALTURA_SPRITE / 2);



    // Coordenadas para a sombra (centralizada na base do sprite)

    int x_sombra = x_atual - (SHADOW_WIDTH / 2);
```

```
// Colocamos a sombra um pouco abaixo do centro do sprite

int y_sombra = y_atual + (ALTURA_SPRITE / 2) - SHADOW_HEIGHT;



// -------------------------------------------------------

// A. DESENHO DA SOMBRA (Primeiro, para ficar abaixo do sprite)

// -------------------------------------------------------



// 1. Salva o Composite atual para restaurá-lo depois

Composite originalComposite = g2d.getComposite();



// 2. Cria um novo AlphaComposite com a opacidade desejada

AlphaComposite alpha = AlphaComposite.getInstance(AlphaComposite.SRC_OVER,
SHADOW_OPACITY);

g2d.setComposite(alpha);



// 3. Define a cor preta e desenha a elipse (sombra)
```

```
g2d.setColor(Color.BLACK);

g2d.fillOval(x_sombra, y_sombra, SHADOW_WIDTH, SHADOW_HEIGHT);



// 4. Restaura o Composite original (para desenhar o sprite 100% opaco)

g2d.setComposite(originalComposite);



// -------------------------------------------------------

// B. DESENHO DO SPRITE DO PINGUIM

// -------------------------------------------------------



if (spritePinguim != null) {

    // Desenha a imagem na tela (100% opaca, pois restauramos o Composite)

    g2d.drawImage(spritePinguim,

                x_sprite,

                y_sprite,
```

```
                LARGURA_SPRITE,

                ALTURA_SPRITE,

                this);

        } else {

            // Fallback (círculo azul)

            g2d.setColor(Color.BLUE);

            g2d.fillOval(x_sprite, y_sprite, LARGURA_SPRITE, ALTURA_SPRITE);

            g2d.setColor(Color.BLACK);

            g2d.drawString("Pinguim (Sem Sprite)", x_sprite, y_sprite - 5);

        }

    }



    // --- MÉTODOS MOUSELISTENER E ACTIONLISTENER (MANTIDOS
INALTERADOS) ---

    @Override

    public void mouseClicked(MouseEvent e) {
```

```
    int novoX = e.getX();

    int novoY = e.getY();

    pinguim.setXAlvo(novoX);

    pinguim.setYAlvo(novoY);

    System.out.println("Alvo definido para: (" + novoX + ", " + novoY + ")");

}



@Override public void mousePressed(MouseEvent e) {}

@Override public void mouseReleased(MouseEvent e) {}

@Override public void mouseEntered(MouseEvent e) {}

@Override public void mouseExited(MouseEvent e) {}



@Override

public void actionPerformed(java.awt.event.ActionEvent e) {

    pinguim.atualizarPosicao();
```

```
        repaint();

    }

}

```

Agora, ao executar, você verá o *sprite* do pinguim com uma sombra escura e ligeiramente transparente na base, que se move suavemente junto com ele.

Prompt: Você deve ter notado que em pinguim possuímos o atributo esta Dançando esse atributo, isso representa mais uma funcionalidade que nosso jogo terá ao apertar uma tecla, como por exemplo "d" o sprite do nosso pinguim ira girar 360° em loop, durante esse processo o nosso pinguim não podera se mover

Resposta: Entendido\! A implementação de um movimento de rotação (dança) em *loop* controlado por uma tecla, enquanto bloqueia o movimento, é uma excelente funcionalidade de jogo.

Para isso, faremos as seguintes alterações:

1. **`PinguimRender`**: Adicionar a lógica de rotação e bloqueio de movimento.

2. **`PainelJogo`**:

* Implementar o `KeyListener` para detectar o pressionamento da tecla **`D`**.

* Ajustar a função de desenho (`paintComponent`) para aplicar a transformação de rotação quando o pinguim estiver dançando.

* Bloquear a atualização da posição se o pinguim estiver dançando.

1\. Atualização da Classe `PinguimRender`

Adicionaremos um estado de ângulo de rotação e modificaremos o método `atualizarPosicao()`.

```java

package cliente;

import java.io.*;

import compartilhado.Ponto2D;

public class PinguimRender extends compartilhado.Pinguim {
```

```
private transient Ponto2D posicaoAtual;

private final double VELOCIDADE_DE_MOVIMENTO = 0.1;



// **NOVO**: Angulo de rotação atual em graus

private transient double anguloRotacao = 0;

// **NOVO**: Velocidade de rotação (graus por frame)

private final double VELOCIDADE_ROTACAO = 10.0;



public PinguimRender(compartilhado.Pinguim pinguim) {

    super(pinguim.getId(),

    pinguim.getXAlvo(),

    pinguim.getYAlvo(),

    pinguim.getDancando(),

    pinguim.getCor()

    );
```

```
    this.posicaoAtual = new Ponto2D(pinguim.getXAlvo(), pinguim.getYAlvo());

    this.anguloRotacao = 0; // Inicializa o ângulo

}



// Getters e Setters (mantidos)

public int getXAtual(){ return this.posicaoAtual.getX(); }

public int getYAtual(){ return this.posicaoAtual.getY(); }

public void setXAtual(int x){ this.posicaoAtual.setX(x); }

public void setYAtual(int y){ this.posicaoAtual.setY(y); }



// **NOVO**: Getter para o ângulo

public double getAnguloRotacao() {

    return this.anguloRotacao;

}
```

```
/**
 * Move suavemente a posicaoAtual em direção à posicaoAlvo,
 * mas apenas se o pinguim não estiver dançando.
 */
public void atualizarPosicao() {
    // Se estiver dançando, não se move, mas atualiza a rotação
    if (super.getDancando()) {
        this.atualizarRotacao();
        return;
    }

    // Lógica de Movimentação (MANTIDA DO EXEMPLO ANTERIOR)
    double currentX = this.posicaoAtual.getX();
    double currentY = this.posicaoAtual.getY();
```

```
int targetX = super.getXAlvo();

int targetY = super.getYAlvo();



double dx = targetX - currentX;

double dy = targetY - currentY;



if (Math.abs(dx) < 1 && Math.abs(dy) < 1) {

    this.posicaoAtual.setX(targetX);

    this.posicaoAtual.setY(targetY);

    return;

}



double moveX = dx * VELOCIDADE_DE_MOVIMENTO;

double moveY = dy * VELOCIDADE_DE_MOVIMENTO;
```

```
    this.posicaoAtual.setX((int) (currentX + moveX));

    this.posicaoAtual.setY((int) (currentY + moveY));

}



/**

 * **NOVO**: Atualiza o ângulo de rotação (chamado quando está dançando)

 */

private void atualizarRotacao() {

    // Incrementa o ângulo

    this.anguloRotacao += VELOCIDADE_ROTACAO;



    // Mantém o ângulo entre 0 e 360 graus (loop)

    if (this.anguloRotacao >= 360) {

        this.anguloRotacao -= 360;

    }
```

```
    }

}
```

2\. Atualização da Classe `PainelJogo`

Implementaremos o `KeyListener` para alternar o estado de dança e a rotação para o desenho.

```java
package principal;

import cliente.PinguimRender;

import java.awt.*;

import java.awt.event.*; // Importa todas as classes de eventos

import javax.swing.*;
```

```
class PainelJogo extends JPanel implements MouseListener, ActionListener, KeyListener {

    // **NOVO**: Implementa KeyListener



    // ... (atributos mantidos) ...

    private PinguimRender pinguim;

    private Image spritePinguim;

    private final int LARGURA_SPRITE = 64;

    private final int ALTURA_SPRITE = 64;

    private final int SHADOW_WIDTH = (int) (LARGURA_SPRITE * 0.8);

    private final int SHADOW_HEIGHT = (int) (ALTURA_SPRITE * 0.25);

    private final float SHADOW_OPACITY = 0.4f;

    private Timer gameTimer;

    private final int FPS = 60;

    private final int DELAY_MS = 1000 / FPS;
```

```
public PainelJogo(PinguimRender pinguim, String caminhoSprite) {

    this.pinguim = pinguim;

    // ... (código de carregamento do sprite) ...



    setBackground(Color.WHITE);

    this.addMouseListener(this);



    // **NOVO**: Adiciona o KeyListener ao painel

    this.addKeyListener(this);



    // **IMPORTANTE**: Faz o painel ser foco, senão o KeyListener não funciona

    this.setFocusable(true);

    this.requestFocusInWindow();
```

```
    gameTimer = new Timer(DELAY_MS, this);

    gameTimer.start();

}



@Override

protected void paintComponent(Graphics g) {

    super.paintComponent(g);



    Graphics2D g2d = (Graphics2D) g;



    int x_atual = pinguim.getXAtual();

    int y_atual = pinguim.getYAtual();



    int x_sprite = x_atual - (LARGURA_SPRITE / 2);

    int y_sprite = y_atual - (ALTURA_SPRITE / 2);
```

```
// Coordenadas para a sombra (centralizada na base do sprite)

int x_sombra = x_atual - (SHADOW_WIDTH / 2);

int y_sombra = y_atual + (ALTURA_SPRITE / 2) - SHADOW_HEIGHT;



// Salva a transformação original para restaurar depois

AffineTransform originalTransform = g2d.getTransform();



// -------------------------------------------------------

// A. DESENHO DA SOMBRA (Não deve rotacionar)

// -------------------------------------------------------

Composite originalComposite = g2d.getComposite();

AlphaComposite alpha = AlphaComposite.getInstance(AlphaComposite.SRC_OVER,
SHADOW_OPACITY);

g2d.setComposite(alpha);

g2d.setColor(Color.BLACK);
```

```
g2d.fillOval(x_sombra, y_sombra, SHADOW_WIDTH, SHADOW_HEIGHT);

g2d.setComposite(originalComposite);



// -------------------------------------------------------

// B. ROTAÇÃO E DESENHO DO SPRITE DO PINGUIM

// -------------------------------------------------------



if (spritePinguim != null) {



    if (pinguim.getDancando()) {

        // 1. Translada o sistema de coordenadas para o centro do pinguim

        g2d.translate(x_atual, y_atual);



        // 2. Rotaciona o sistema de coordenadas pelo ângulo atual

        double radianos = Math.toRadians(pinguim.getAnguloRotacao());
```

```
            g2d.rotate(radianos);



            // 3. Desenha a imagem, compensando a translação para o centro (desenha em
-metade, -metade)

            g2d.drawImage(spritePinguim,

                    -LARGURA_SPRITE / 2,

                    -ALTURA_SPRITE / 2,

                    LARGURA_SPRITE,

                    ALTURA_SPRITE,

                    this);



            // 4. Restaura a transformação original após desenhar

            g2d.setTransform(originalTransform);



        } else {

            // Desenho normal (sem rotação)
```

```
            g2d.drawImage(spritePinguim,
                        x_sprite,
                        y_sprite,
                        LARGURA_SPRITE,
                        ALTURA_SPRITE,
                        this);
        }
    }
    // ... (código fallback) ...
}


// *************** MÉTODOS DE CONTROLE DE EVENTOS ***************


@Override
public void mouseClicked(MouseEvent e) {
```

```
    // Só permite clique (e mudança de alvo) se não estiver dançando
    if (!pinguim.getDancando()) {
        int novoX = e.getX();
        int novoY = e.getY();
        pinguim.setXAlvo(novoX);
        pinguim.setYAlvo(novoY);
        System.out.println("Alvo definido para: (" + novoX + ", " + novoY + ")");
    } else {
         System.out.println("Pinguim está dançando e não pode se mover!");
    }
}

@Override
public void actionPerformed(java.awt.event.ActionEvent e) {
    // Atualiza a posição (que também cuida da rotação se estiver dançando)
```

```
    pinguim.atualizarPosicao();

    repaint();

}



// *************** MÉTODOS DA INTERFACE KEYLISTENER ***************



@Override

public void keyPressed(KeyEvent e) {

    // Detecta a tecla 'D' (case insensitive)

    if (e.getKeyChar() == 'd' || e.getKeyChar() == 'D') {

        // Alterna o estado de dança

        pinguim.setDancando(!pinguim.getDancando());

        System.out.println("Estado de dança alterado para: " + pinguim.getDancando());

    }
```

```
        // Se precisar que o painel receba o foco novamente (opcional, mas bom ter)
        requestFocusInWindow();
    }

    // Os outros métodos de KeyListener ficam vazios, pois só precisamos do keyPressed
    @Override public void keyTyped(KeyEvent e) {}
    @Override public void keyReleased(KeyEvent e) {}

    // ... (Outros métodos MouseListener permanecem vazios...)
    @Override public void mousePressed(MouseEvent e) {}
    @Override public void mouseReleased(MouseEvent e) {}
    @Override public void mouseEntered(MouseEvent e) {}
    @Override public void mouseExited(MouseEvent e) {}
}
```

O que foi feito:

1. **Bloqueio de Movimento**: No `mouseClicked` e no `atualizarPosicao()`, há uma verificação em `pinguim.getDancando()`. Se for `true`, o movimento é ignorado.

2. **Toggle de Dança**: O `keyPressed` verifica a tecla 'D' e chama `pinguim.setDancando()` para alternar o estado booleano.

3. **Rotação**: No `paintComponent`, usamos `Graphics2D` para:

   * **Salvar o Estado Original**: `g2d.getTransform()` é crucial para que o desenho da sombra (e o próximo frame) não seja afetado pela rotação.

   * **Translação**: Movemos a origem (`0,0`) para o centro do pinguim (`x_atual`, `y_atual`).

   * **Rotação**: Aplicamos a rotação de `pinguim.getAnguloRotacao()`.

   * **Desenho**: Desenhamos a imagem em coordenadas negativas (`-LARGURA_SPRITE / 2`), pois a origem está no centro do objeto.

   * **Restauração**: Restauramos a transformação original (`g2d.setTransform(originalTransform)`).

Execute o programa, clique para mover e aperte **`D`** para iniciar e parar a dança de rotação\!

Prompt: Eu gostaria que agora, invés de ao apertar a tecla "d" o pinguim iniciasse a dança, que ao pressionar um botão com o sprite de nome botaoDanca a dança fosse iniciada, esse botão estaria no canto inferior direito da tela e seria relativamente pequeno.

Resposta: Essa é uma ótima maneira de melhorar a interface do usuário\! Substituir o evento de teclado por um botão é uma prática comum em aplicações gráficas.

Para implementar o botão que inicia e para a dança no canto inferior direito, faremos as seguintes modificações:

1. **Criação do Botão:** Utilizaremos um `JButton` e definiremos seu ícone com o *sprite* `botaoDanca.png`.

2. **Layout:** Usaremos um `BorderLayout` no `JFrame` principal e posicionaremos o `PainelJogo` no `CENTER` e o botão em um novo painel auxiliar posicionado no `SOUTH` ou `EAST`. O ideal para o canto inferior é usar um `JPanel` com `FlowLayout` alinhado à direita.

3. **Ação:** O `ActionListener` do botão será responsável por alternar o estado de dança do pinguim.

4. **Remoção:** Removemos o `KeyListener` e a lógica de tecla da classe `PainelJogo`.

1\. Atualização da Classe `PainelJogo`

A principal mudança é **remover toda a lógica de `KeyListener`** e os métodos relacionados.

`PainelJogo.java` (Resumido nas alterações):

```java
package principal;

// ... imports ...
import cliente.PinguimRender;
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;
```

```
// A classe NÃO implementa mais KeyListener

class PainelJogo extends JPanel implements MouseListener, ActionListener {

    // ... (atributos mantidos) ...



    public PainelJogo(PinguimRender pinguim, String caminhoSprite) {

        this.pinguim = pinguim;

        // ... (código de carregamento do sprite) ...



        setBackground(Color.WHITE);

        this.addMouseListener(this);



        // **REMOVIDO**: this.addKeyListener(this);

        // **REMOVIDO**: this.setFocusable(true);

        // **REMOVIDO**: this.requestFocusInWindow();
```

```
    gameTimer = new Timer(DELAY_MS, this);

    gameTimer.start();

}



// ... (método paintComponent mantido inalterado) ...



@Override

public void mouseClicked(MouseEvent e) {

    // **IMPORTANTE**: O foco não é mais necessário aqui

    // requestFocusInWindow();



    if (!pinguim.getDancando()) {

        // ... (lógica de movimento) ...

    } else {

         System.out.println("Pinguim está dançando e não pode se mover!");
```

```
        }

    }



    // ... (actionPerformed e outros MouseListener mantidos) ...



    // **REMOVIDOS** os métodos keyTyped, keyPressed, keyReleased

}
```

2\. Atualização da Classe `JogoPinguim` (Integração da UI)

Esta classe fará a maior parte do trabalho de montagem do `JFrame`, adicionando o botão e configurando sua ação.

`JogoPinguim.java`:

```java

package principal;



import compartilhado.Pinguim;

import cliente.PinguimRender;

import java.awt.*;

import javax.swing.*;

import java.awt.event.ActionEvent;

import java.awt.event.ActionListener;

import java.io.File;



public class JogoPinguim {



    public static void main(String[] args) {

        SwingUtilities.invokeLater(() -> {
```

```
// 1. Instância do Pinguim

compartilhado.Pinguim basePinguim = new Pinguim(1, 400, 300, false, "Preto");

PinguimRender pinguimRender = new PinguimRender(basePinguim);



// 2. Configuração da Janela Principal (JFrame)

JFrame frame = new JFrame("Pinguim Game (Voltante Edition)");

frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

frame.setSize(800, 600);

frame.setLocationRelativeTo(null);



// Define o layout do frame como BorderLayout

frame.setLayout(new BorderLayout());



// 3. Criação e Adição do Painel do Jogo
```

```
        PainelJogo painelJogo = new PainelJogo(pinguimRender, "voltante.png");

        // Adiciona o painel de jogo no centro

        frame.add(painelJogo, BorderLayout.CENTER);



        // 4. Criação do Botão de Dança



        // Tenta carregar o sprite do botão

        ImageIcon iconDanca = null;

        try {

            // Tenta carregar via Classpath (caminho relativo à raiz do Classpath)

            iconDanca = new
ImageIcon(JogoPinguim.class.getClassLoader().getResource("botaoDanca.png"));

        } catch (Exception e) {

            System.err.println("Erro ao carregar sprite do botão (botaoDanca.png). Usando
texto.");

        }
```

```
JButton btnDanca = new JButton();

if (iconDanca != null && iconDanca.getImageLoadStatus() ==
MediaTracker.COMPLETE) {

    // Redimensiona o ícone se necessário (exemplo: para 48x48)

    Image img = iconDanca.getImage().getScaledInstance(48, 48,
Image.SCALE_SMOOTH);

    btnDanca.setIcon(new ImageIcon(img));

    btnDanca.setPreferredSize(new Dimension(50, 50)); // Define um tamanho
pequeno

    btnDanca.setText(""); // Remove o texto se houver ícone

} else {

    btnDanca.setText("Dançar");

    btnDanca.setPreferredSize(new Dimension(100, 50));

}

btnDanca.setToolTipText("Alternar estado de dança do pinguim");



// 5. Criação do Painel de UI (para posicionar o botão)
```

```
JPanel uiPanel = new JPanel();

// Usa FlowLayout alinhado à direita

uiPanel.setLayout(new FlowLayout(FlowLayout.RIGHT));

uiPanel.setOpaque(false); // Torna o painel transparente para ver o painel do jogo



uiPanel.add(btnDanca);



// 6. Adiciona o Painel de UI no canto inferior direito (SOUTH_EAST)

// Em BorderLayout, usar BorderLayout.SOUTH coloca o painel na parte inferior.

frame.add(uiPanel, BorderLayout.SOUTH);



// 7. Configuração da Ação do Botão

btnDanca.addActionListener(new ActionListener() {

    @Override
```

```
            public void actionPerformed(ActionEvent e) {

                // Alterna o estado de dança do PinguimRender

                boolean estaDancando = pinguimRender.getDancando();

                pinguimRender.setDancando(!estaDancando);




                System.out.println("Estado de dança alterado pelo botão para: " +
pinguimRender.getDancando());

            }

        });




        // 8. Exibir a Janela

        frame.setVisible(true);

    });

  }

}

```

Passo a Passo para o Novo Sprite

Para que este código funcione, certifique-se de que o novo arquivo de *sprite* do botão, **`botaoDanca.png`**, esteja na mesma pasta do *Classpath* que o `voltante.png`.

1. **Caminho do Sprite:** Coloque o arquivo **`botaoDanca.png`** na sua pasta de recursos (ex: `resources/` ou a raiz do `src`).

2. **Execução:** Compile e execute a classe `JogoPinguim`.

O botão aparecerá na parte inferior direita da tela. Ao clicar nele, o estado de dança do pinguim será alternado. Enquanto dança, o clique do mouse no `PainelJogo` será ignorado.

Prompt: package compartilhado;

```
import java.io.*;


public class Pinguim implements Serializable{

    private int id;
```

```
    // private Ponto2D posicaoAtual

    private String username;

    private Ponto2D posicaoAlvo; //Para o servidor Alvo é a posição atual

    private boolean estaDancando;

    private String cor;




    public Pinguim(int id, String username, int x_Alvo, int y_Alvo, boolean estaDancando,
String cor){

        this.id = id;

        this.username = username;

        // this.posicaoAtual = new Ponto2D(x_Atual, y_Atual);

        this.posicaoAlvo = new Ponto2D(x_Alvo, y_Alvo);

        this.estaDancando = estaDancando;

        this.cor = cor;

    }
```

```
public int getId(){

    return this.id;

}



public void setId(int id){

    this.id = id;

}



public String getUsername()

{

    return this.username;

}



public void setUsername(String username)

{
```

```
    this.username = username;

}


public boolean getDancando(){

    return this.estaDancando;

}


public void setDancando(boolean dancando){

    this.estaDancando = dancando;

}


public void dancar(){

    this.estaDancando = !this.estaDancando;

}
```

```
public String getCor(){

    return this.cor;

}



public void setCor(String cor){

    this.cor = cor;

}



// public int getXAtual(){

//  return posicaoAtual.getX();

// }



// public int getYAtual(){

//  return posicaoAtual.getY();

// }
```

```
public int getXAlvo(){

    return posicaoAlvo.getX();

}



public int getYAlvo(){

    return posicaoAlvo.getY();

}



// public void setXAtual(int x){

//  this.posicaoAtual.setX(x);

// }



// public void setYAtual(int y){

//  this.posicaoAtual.setY(y);
```

```
    // }



    public void setXAlvo(int x){

        this.posicaoAlvo.setX(x);

    }



    public void setYAlvo(int y){

        this.posicaoAlvo.setY(y);

    }

}
```

Fiz uma pequena alteração na classe pinguim, pois agora ela possuí o atributo de Username, gostaria que no jogo esse username aparecesse em cima do sprite do pinguim em letras pretas e pequenas.

Resposta: Ótima adição\! Incluir o `username` na classe `Pinguim` é fundamental para um jogo multiusuário.

Para que o nome de usuário apareça em letras pretas e pequenas acima do *sprite* do pinguim, faremos uma pequena modificação na lógica de desenho (`paintComponent`) na classe `PainelJogo`.

Atualização da Classe `PinguimRender`

Como você alterou o construtor da classe base (`Pinguim`), precisamos garantir que o construtor de `PinguimRender` também seja atualizado para aceitar e passar o `username`.

`PinguimRender.java` (Apenas o Construtor Modificado):

```java

package cliente;

import java.io.*;

import compartilhado.Ponto2D;

// ... outros imports ...
```

```
public class PinguimRender extends compartilhado.Pinguim {



    private transient Ponto2D posicaoAtual;

    private final double VELOCIDADE_DE_MOVIMENTO = 0.1;

    private transient double anguloRotacao = 0;

    private final double VELOCIDADE_ROTACAO = 10.0;



    // Construtor atualizado para receber o username

    public PinguimRender(int id, String username, int x_Alvo, int y_Alvo, boolean
estaDancando, String cor) {

        super(id, username, x_Alvo, y_Alvo, estaDancando, cor);

        this.posicaoAtual = new Ponto2D(x_Alvo, y_Alvo);

        this.anguloRotacao = 0;

    }



    // Construtor de cópia (útil para serialização/rede)
```

```
    public PinguimRender(compartilhado.Pinguim pinguim) {

        super(pinguim.getId(),

        pinguim.getUsername(), // **NOVO**: Passa o username

        pinguim.getXAlvo(),

        pinguim.getYAlvo(),

        pinguim.getDancando(),

        pinguim.getCor()

        );



        this.posicaoAtual = new Ponto2D(pinguim.getXAlvo(), pinguim.getYAlvo());

        this.anguloRotacao = 0;

    }



    // ... (restante da classe PinguimRender inalterado) ...

}
```

```

---

## 2\. Atualização da Classe `PainelJogo` (Desenho do Username)

Adicionaremos a lógica de desenho do texto no método `paintComponent`. Usaremos `Graphics2D` para configurar a fonte e garantir que a rotação do pinguim não afete o texto.

**`PainelJogo.java` (Modificações no `paintComponent`):**

```java
// ... imports ...

class PainelJogo extends JPanel implements MouseListener, ActionListener {

    // ... atributos e construtor inalterados ...
```
```

```
@Override

protected void paintComponent(Graphics g) {

    super.paintComponent(g);



    Graphics2D g2d = (Graphics2D) g;



    int x_atual = pinguim.getXAtual();

    int y_atual = pinguim.getYAtual();



    // As coordenadas de desenho para o sprite (centro)

    int x_sprite = x_atual - (LARGURA_SPRITE / 2);

    int y_sprite = y_atual - (ALTURA_SPRITE / 2);



    // ... (Cálculo e Desenho da Sombra - MANTIDO INALTERADO) ...
```

```
// Salva a transformação original ANTES do desenho do Pinguim/Username

AffineTransform originalTransform = g2d.getTransform();



// -------------------------------------------------------

// A. DESENHO DA SOMBRA (Primeiro, para ficar abaixo do sprite)

// ... (código da sombra) ...

// -------------------------------------------------------



// -------------------------------------------------------

// B. ROTAÇÃO E DESENHO DO SPRITE DO PINGUIM (MANTIDO INALTERADO)

// -------------------------------------------------------



if (spritePinguim != null) {
```

```
if (pinguim.getDancando()) {

    // Lógica de rotação...

    g2d.translate(x_atual, y_atual);

    double radianos = Math.toRadians(pinguim.getAnguloRotacao());

    g2d.rotate(radianos);



    g2d.drawImage(spritePinguim,

                -LARGURA_SPRITE / 2,

                -ALTURA_SPRITE / 2,

                LARGURA_SPRITE,

                ALTURA_SPRITE,

                this);



    // Restaura a transformação antes de desenhar o nome

    g2d.setTransform(originalTransform);
```

```
} else {

    // Desenho normal (sem rotação)

    g2d.drawImage(spritePinguim,

                x_sprite,

                y_sprite,

                LARGURA_SPRITE,

                ALTURA_SPRITE,

                this);

}



// -------------------------------------------------------

// C. DESENHO DO USERNAME (Fora do bloco de rotação)

// -------------------------------------------------------
```

```
String username = pinguim.getUsername();



// 1. Define a fonte pequena e preta

g2d.setColor(Color.BLACK);

g2d.setFont(new Font("Arial", Font.BOLD, 12));



// 2. Obtém as métricas da fonte para centralizar o texto

FontMetrics fm = g2d.getFontMetrics();

int larguraTexto = fm.stringWidth(username);



// 3. Calcula a posição X para centralizar o texto horizontalmente

int x_texto = x_atual - (larguraTexto / 2);



// 4. Calcula a posição Y para colocar o texto acima do sprite

// Definimos uma margem de 10 pixels acima do topo do sprite.
```

```
            int y_texto = y_sprite - 10;



            // 5. Desenha o username

            g2d.drawString(username, x_texto, y_texto);

        }

        // ... (código fallback inalterado) ...

    }

    // ... (restante da classe inalterado) ...

}
```

3\. Atualização da Classe `JogoPinguim`

Por fim, precisamos atualizar a chamada de criação da instância para fornecer o novo parâmetro `username`.

`JogoPinguim.java` (Modificação no `main`):

```java
package principal;

import compartilhado.Pinguim;
import cliente.PinguimRender;
// ... outros imports ...

public class JogoPinguim {

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
```

```
        // Defina o username aqui

        String nomeUsuario = "PenguinMaster7";



        // 1. Instância do Pinguim com o novo parâmetro username

        // public Pinguim(int id, String username, int x_Alvo, int y_Alvo, boolean
estaDancando, String cor)

        compartilhado.Pinguim basePinguim = new Pinguim(1, nomeUsuario, 400, 300, false,
"Preto");



        // 2. Cria a instância de renderização/movimentação a partir da bases

        PinguimRender pinguimRender = new PinguimRender(basePinguim);



        // ... (restante do código de configuração da janela e botão inalterado) ...

    });

  }

}
```

```

Ao executar agora, o nome `"PenguinMaster7"` (ou o nome que você definir) aparecerá centralizado, em preto e em uma fonte pequena, logo acima do seu pinguim, e não será afetado pela rotação de dança.
```